

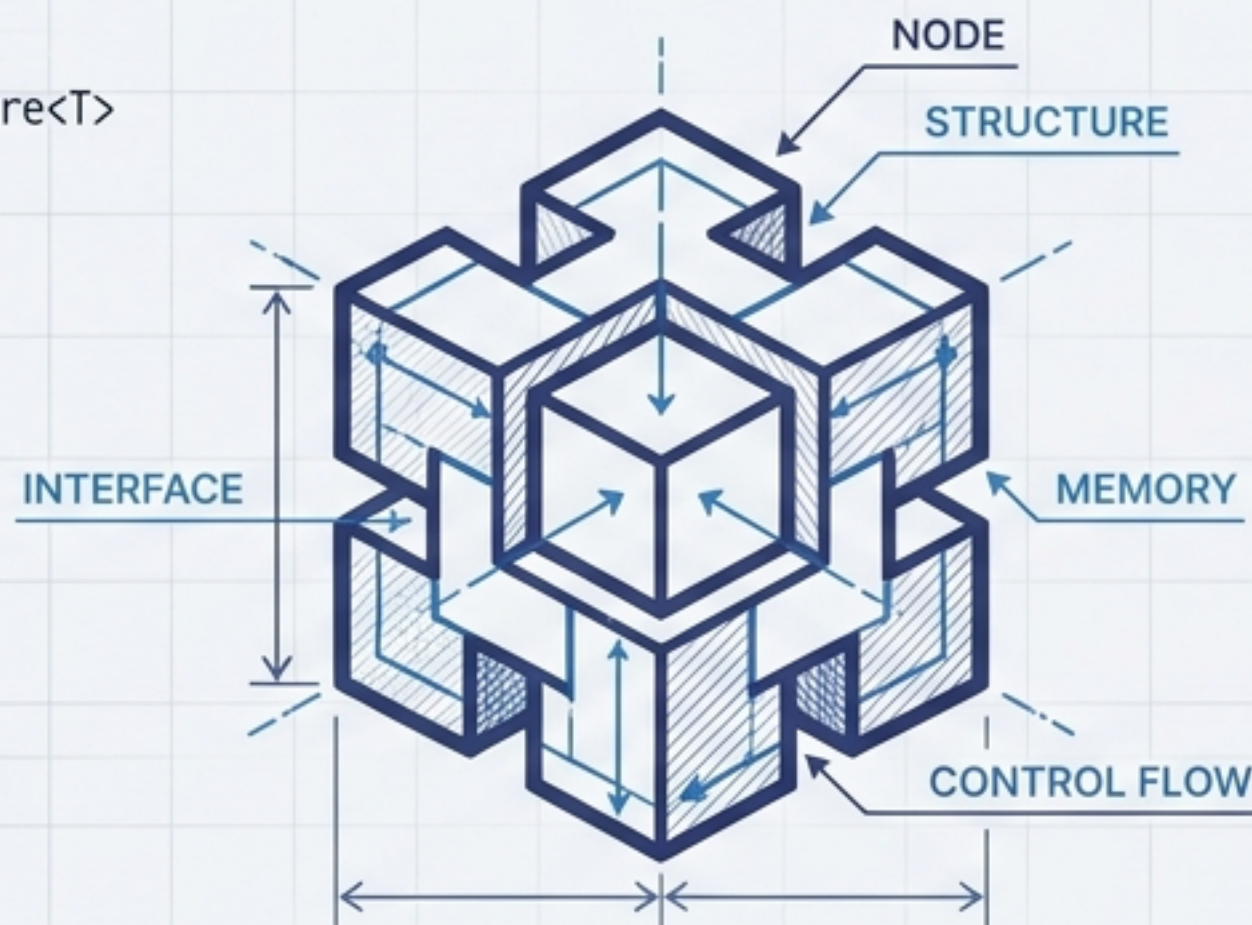
```

public interface IDataStructure<T>
{
    public Idotaxurt;
    ...
}

struct MemoryBlock {
    unsafe fixed byte* ptr;
}

abstract class NodeBase
{
    ...
}

```



```

IEnumerator<T> GetEnumerator() {
    ...
}

foreach (var item in list.ReverseSafe())
{
    try {
        ...
    } catch (Exception e) {
        Log.Error(e);
    }
}

public ref struct Span<T>
{
    ...
}

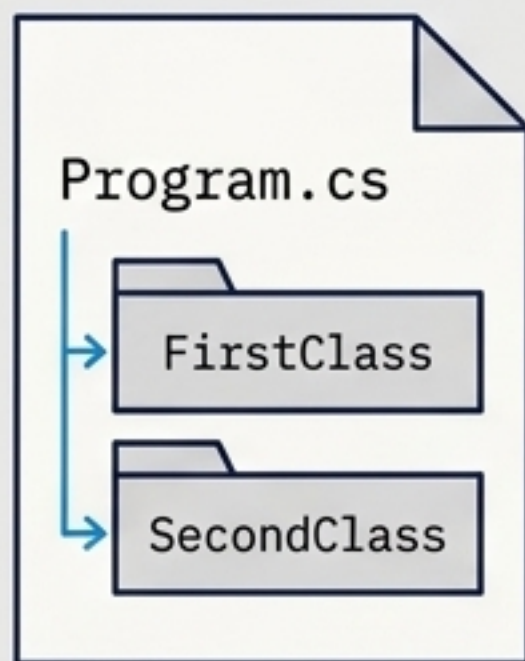
```

C# 데이터 설계의 구조와 제어

추상화부터 메모리 구조, 그리고 리스트의 안전한 역방향 제어까지

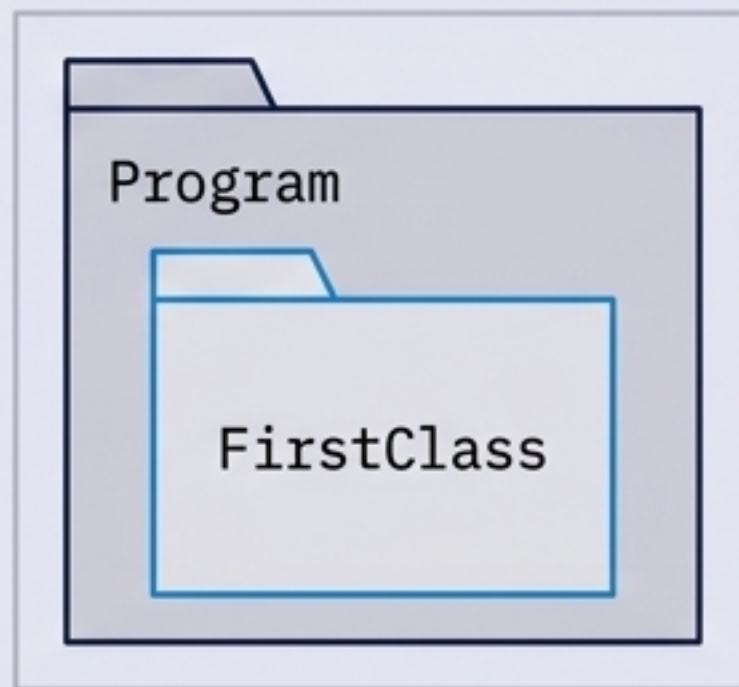
클래스 설계를 배치하는 세 가지 아키텍처

1. 병합 (하나의 파일)



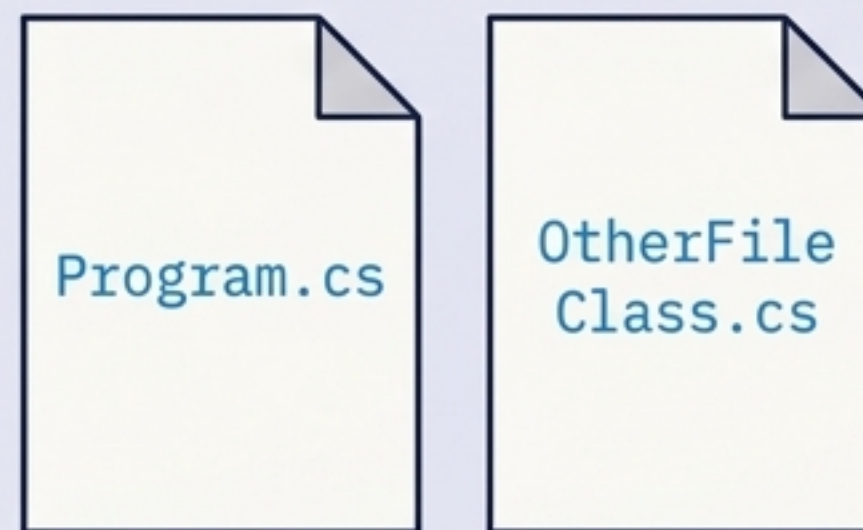
단일 C# 파일 내부에 여러 클래스를 나열하여 배치. 소규모 테스트에 적합.

2. 중첩 (클래스 내부)



특정 클래스 내부에서만 사용할 목적으로 설계를 완전히 종속시킴.

3. 분리 (독립 파일)



클래스명과 동일한 새 .cs 파일을 생성하여 분리. 실무 표준 방식.

IDE를 활용한 설계도 자동 분리 워크플로우

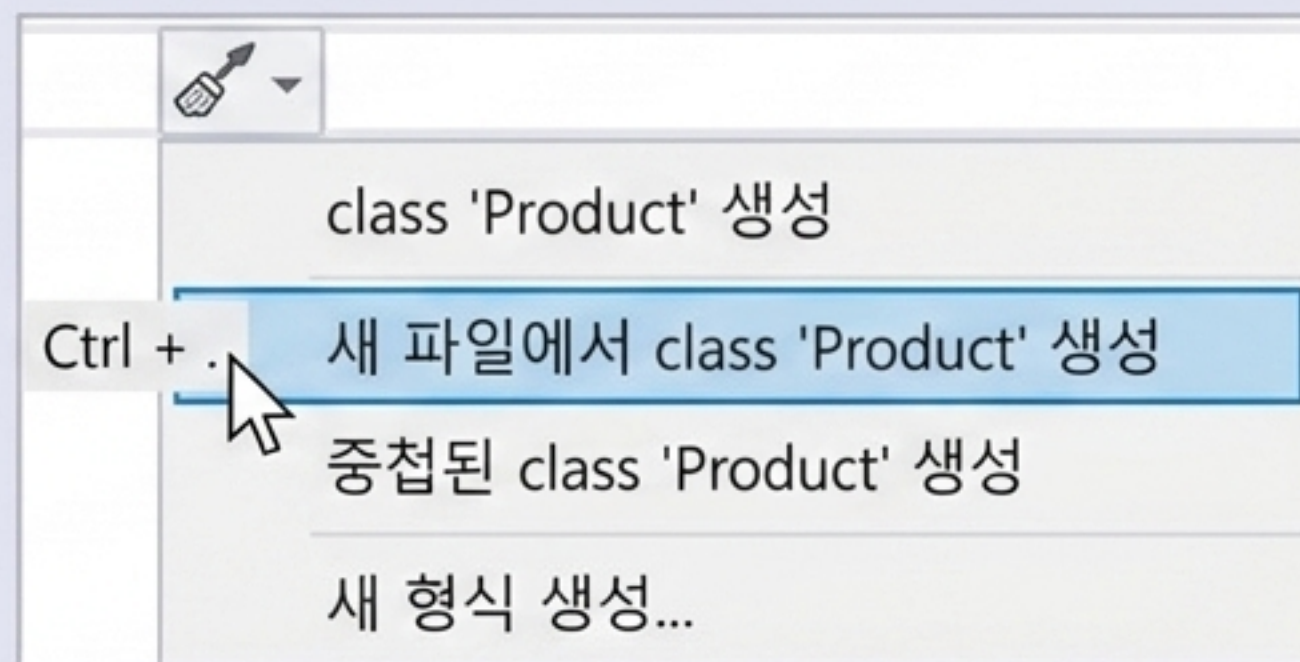
Step 1: 존재하지 않는 클래스 호출

```
Product product = new Product();
```

Main() 메서드에서 아직 만들지 않은 클래스의 인스턴스를 선언.



Step 2: 퀵 액션 (Quick Actions) 실행



단축키를 통해 즉각적으로 새 .cs 파일을 독립 생성하여 작업 속도 극대화.

시스템 기본 식별자를 덮어쓰는 이름 충돌(Name Collision)의 함정

사용자 정의 클래스 선언

```
01 class Program
02 {
03     class Math { }
04
05     static void Main(string[] args)
06     {
07         Console.WriteLine(Math.Abs(-10));
08     }
09 }
```

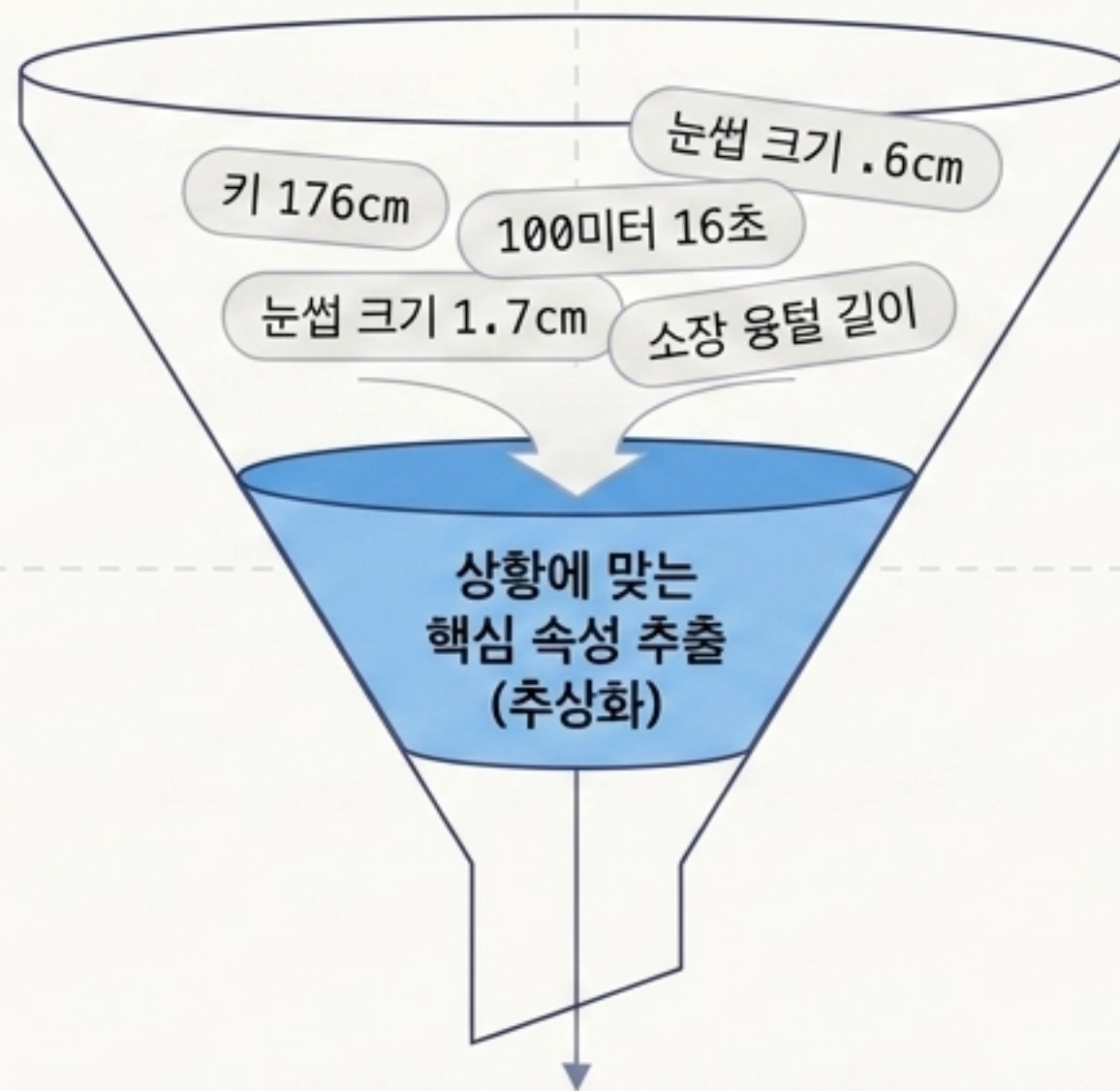
이름 충돌 발생

CS0117: 'Program.Math'에는 'Abs'에 대한 정의가 포함되어 있지 않습니다.

C#은 방대한 내장 클래스를 지원합니다. 사용자가 Math라는 이름의 클래스를 선언하는 순간, 기존 시스템의 Math 클래스 식별자가 가려져 Math.Abs()와 같은 내장 메서드를 전혀 사용할 수 없게 됩니다.

무한한 현실 세계의 데이터를 코드로 정제하는 '추상화'

현실 데이터



모델 클래스

```
public string id;  
public string name;  
public int grade;
```

수많은 속성 중 목적에 부합하는
데이터만 추출하여
객체의 뼈대(Model Class)를 설계합니다.

설계도에 데이터를 담는 두 가지 차원: 인스턴스 vs 클래스 변수

비교 기준	인스턴스 (Instance) 변수	클래스 (Class/Static) 변수
선언 방식	public [자료형] [이름]; (소문자 시작 권장)	public static [자료형] [이름]; (static 키워드)
호출 방식	객체명.변수 (예: product.name)	클래스명.변수 (예: Math.PI)
메모리 할당	객체(인스턴스)가 생성될 때마다 개별적으로 메모리 할당	프로그램 실행 시 메모리에 단 1개만 할당되어 공유됨

인스턴스 예제

```
Product product = new Product();  
product.name
```

클래스 예제

```
public static double PI = 3.141592;  
MyMath.PI
```

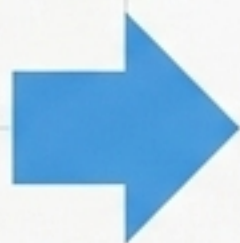

객체의 생성과 동시에 본질을 부여하는 초기화 문법

Before & After

순차적 할당 방식

```
Product product = new Product();  
product.name = "감자";  
product.price = 2000;
```

인스턴스를 먼저 생성한 후,
변수에 순차적으로 접근하여 값을 변경.

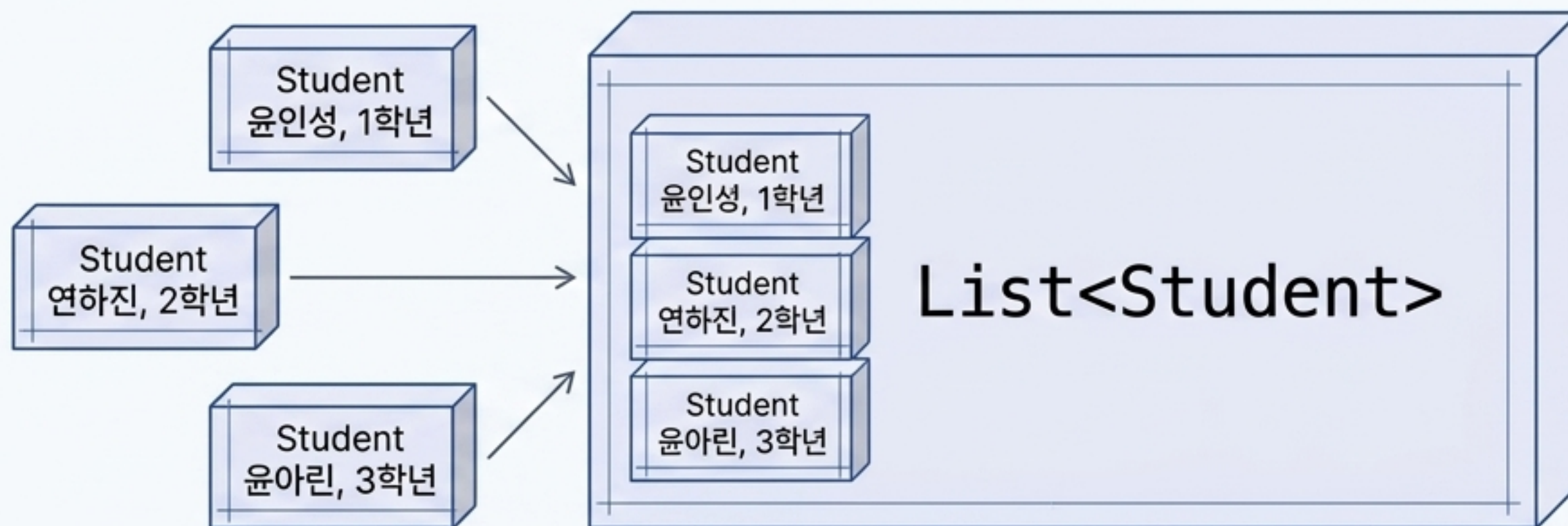


블록 초기화 방식 (권장)

```
Product productA = new Product() {  
    name = "감자",  
    price = 2000  
};
```

중괄호 { }를 사용하여 생성과 동시에
프로퍼티를 직관적으로 초기화.
코드의 **가독성**이 비약적으로 상승합니다.

모델 클래스 군집화: List 컨테이너 구축



```
List<Student> list = new List<Student>() {  
    new Student() { name = "윤인성", grade = 1 },  
    new Student() { name = "연하진", grade = 2 },  
    ...  
};
```

변수만 가지고 있는 데이터의 묶음을
'**모델 클래스(Model Class)**'라고
부르며, 이를 제어하기 위해 **제네릭
List** 와 결합하여 사용합니다.

동적 데이터 제어의 미션: 특정 조건의 요소 제거

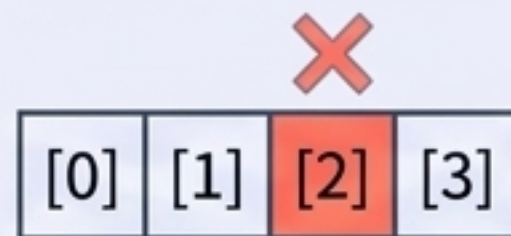
미션: 1학년보다 학년이 높은 학생을 찾아 리스트에서 전부 제거하라.

도구 1: Remove(object element)



특정 객체(값) 자체를 지정하여 리스트에서 찾아 제거.

도구 2: RemoveAt(int index)

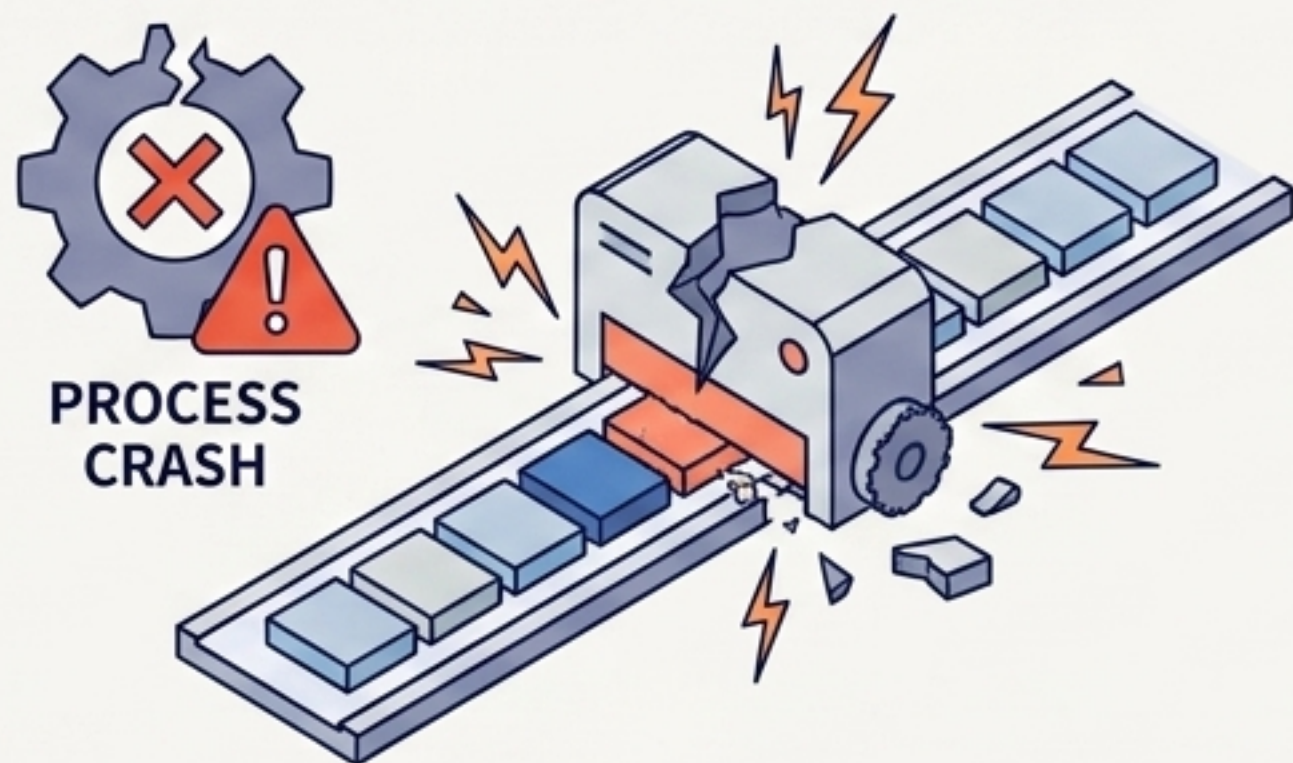


리스트 내의 특정 위치(인덱스 번호)를 기반으로 요소를 핀포인트 제거.

Case 1. foreach 반복문의 치명적 오류

```
foreach (var item in list) {  
    if (item.grade > 1)  
        list.Remove(item);  
}
```

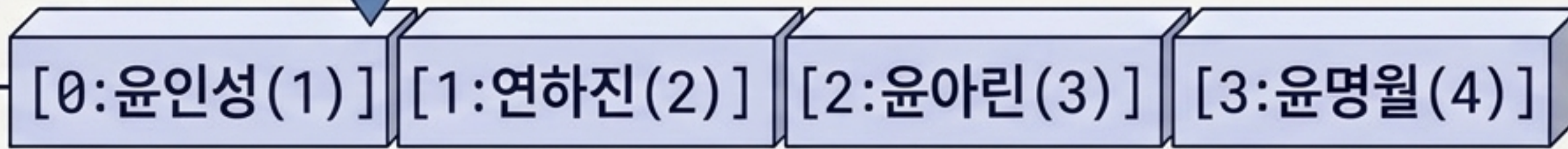
InvalidOperationException: 컬렉션이 수정되었습니다. 열거 작업이 실행되지 않을 수도 있습니다.



foreach 반복문은 내부 구조(맵)를 따라 이동합니다. 도중에 요소가 삭제되어 리스트의 구조가 깨지면, 추적 스캐너가 길을 잃고 즉각적인 예외(Exception)를 발생시키며 프로그램을 강제 종료합니다.

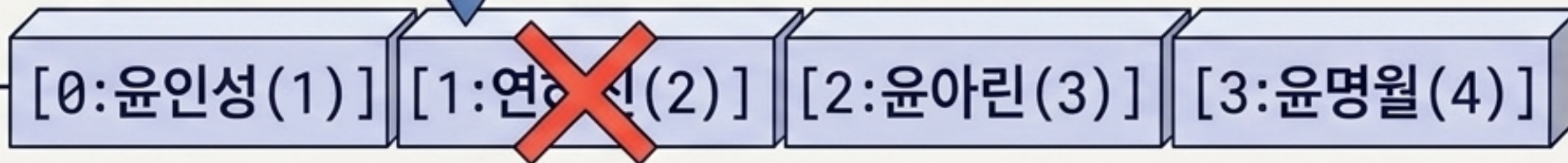
Case 2. 정방향 for 루프의 '인덱스 밀림(Index Shift)' 버그

Step 1 (i=0) 스캐너(i=0)



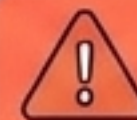
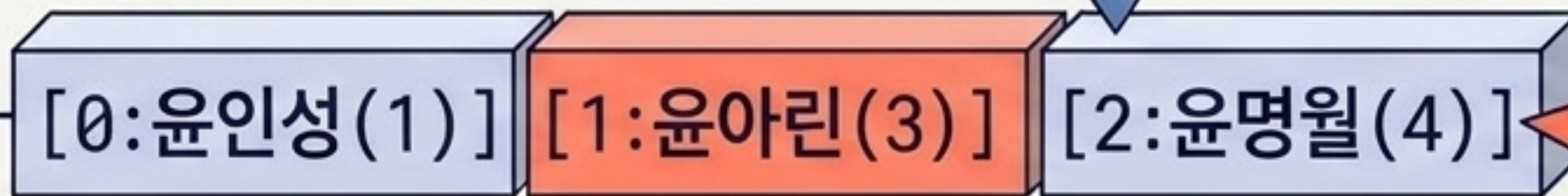
윤인성 유지.

Step 2 (i=1) 스캐너(i=1)



조건(grade>1) 충족.
연하진 제거.

Step 3 (i=2) 스캐너(i=2)



‘윤아린’ 누락!

요소가 지워지며 뒤의 데이터가 앞으로 밀려왔지만, 스캐너(i)는 전진해버려 1번 인덱스로 당겨진 ‘윤아린’을 검사하지 않고 건너뜁니다.

Case 3. 역방향(Reverse) for 루프를 통한 안전한 제어

```
for (int i = list.Count - 1; i >= 0; i--) {  
    if(list[i].grade > 1)  
        list.RemoveAt(i);  
}
```



뒤에서부터 제거하면 앞쪽 인덱스가 밀리지 않아 지반 구조가 무너지지 않음.

실행 결과

윤인성 : 1
구지연 : 1

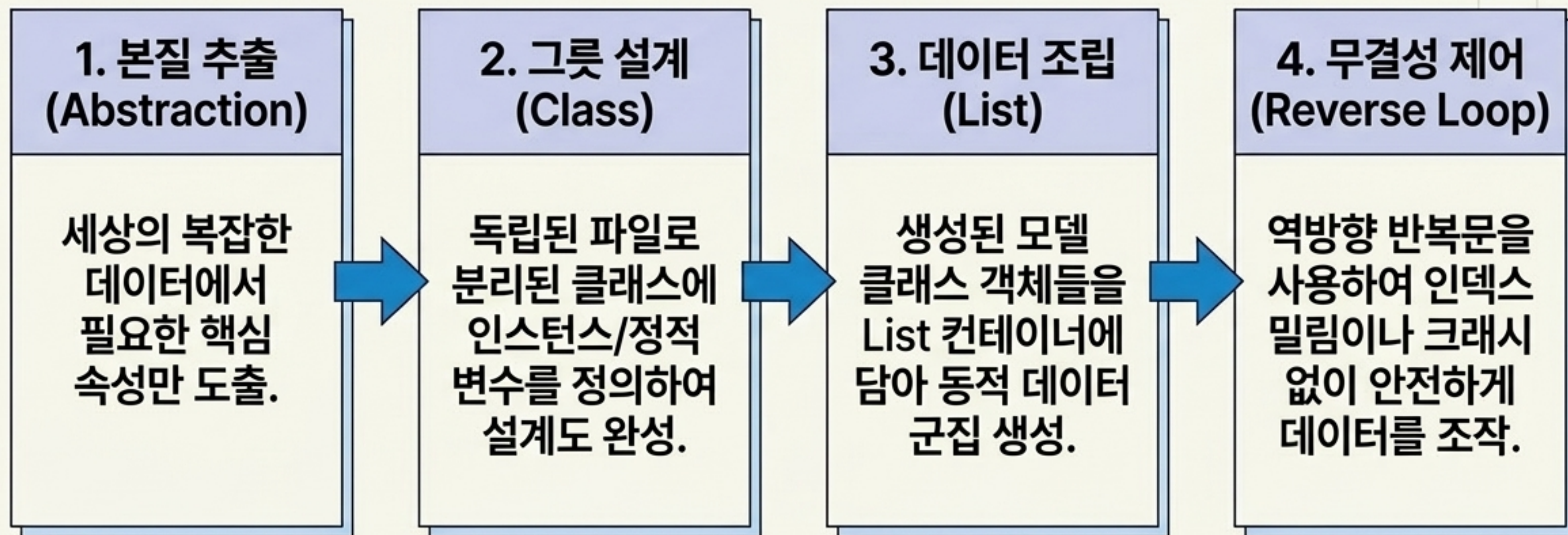
Rule of Thumb

리스트의 요소를 동적으로 제거할 때는 반드시 역(Reverse) for 반복문을 사용하라.

List 반복문 요소 제거 전략 진단표

반복문 종류 (Method)	상태 (Status)	결과/원인 (Result/Cause)
foreach	❌ 시스템 크래시 (Exception)	컬렉션 구조가 도중에 변경되어 열거(Enumeration) 작업이 즉각 중단됨.
정방향 for (i++)	⚠️ 논리적 버그 발생 (Silent Failure)	에러 없이 실행되지만, 삭제 시 인덱스가 앞으로 밀리면서 다음 요소를 검사하지 않고 건너뛰는 치명적 오류 발생.
역방향 for (i--)	✅ 완벽하게 정상 작동 (Best Practice)	뒤에서부터 지우기 때문에 앞쪽 인덱스 배열에 전혀 영향을 주지 않아 누락 없이 모든 요소 검사 가능.

종합: 안전하고 유연한 C# 데이터 관리 워크플로우



코딩은 단순히 문법을 치는 것이 아닙니다. 추상화된 현실의 데이터를 안전한 메모리 구조 속에 배치하고, 예외 없이 통제하는 아키텍처의 설계입니다.